

TeaLeafNet

Tea Leaf Disease Detection System

Implementation 2 — Summary Report

University	Rajarata University of Sri Lanka
Department	Department of Computing
Course	ICT 3212 – Introduction to Intelligent Systems
Project Title	TeaLeafNet: An Image-Based Tea Leaf Disease Detection System
Group Name	Quantum Coders
Submission Date	11 March 2026
GitHub Repo	https://github.com/Piyumi2025/TeaLeafNet

#	Name	Student ID
1	M J H A P Madushani (Team Leader)	ICT/2022/142
2	M K H K Madushani	ICT/2022/107
3	W K D Bhagya	ICT/2022/110
4	M T Rathnayake	ICT/2022/114
5	M G J Sinty	ICT/2022/043

1. Project Overview & Implementation 2 Objectives

In Implementation 1, a baseline CNN model was trained for 6 epochs and achieved a final validation accuracy of 83.12%. Two key limitations were identified from that run:

- Mild overfitting: training accuracy (88.19%) exceeded validation accuracy (83.12%), a gap of 5.07%.
- Poor Bird_Eye_Spot classification: the baseline struggled to distinguish Bird_Eye_Spot from Anthracnose due to their visually similar spot patterns.

The objective of Implementation 2 is to resolve these issues by introducing regularization, data augmentation, and adaptive training strategies. The final model — TeaLeafNet V2 Lite — was trained for the full 30 epochs, achieving a best validation accuracy of 89.61% and a final evaluated accuracy of 87.01%.

The improvements introduced in Implementation 2 are:

- Data Augmentation (Lite)** — RandomFlip, RandomRotation (10%), RandomZoom (10%) applied to training data only.
- Dropout Regularization** — 30% dropout on the Dense layer to combat overfitting.

- **EarlyStopping Callback** — monitors `val_loss` with `patience=5` and restores best weights automatically.
- **ReduceLROnPlateau Callback** — reduces learning rate dynamically when `val_loss` stalls, enabling finer convergence.

2. Dataset Preparation & Preprocessing

2.1 Dataset Source & Split

The same Kaggle Tea Sickness Dataset from Implementation 1 was used with the same 80/20 train/validation split and random seed, ensuring a fair comparison between baseline and V2 results.

Parameter	Value
Dataset Source	Kaggle – Tea Sickness Dataset
Total Images	387 images across 4 classes
Training Set	310 images (80%)
Validation Set	77 images (20%)
Image Size	224 × 224 pixels
Normalization	Rescaling(1./255) — embedded inside model
Random Seed	123 (same as Implementation 1)

2.2 Class Distribution

Class Label	Total Images	Training (~80%)	Validation (~20%)
Algal_Leaf	113	~90	~23
Anthracoze	100	~80	~20
Bird_Eye_Spot	100	~80	~20
Healthy	74	~60	~14
TOTAL	387	310	77

2.3 Data Augmentation Pipeline (Lite)

A 'Lite' augmentation pipeline with gentler parameters was applied exclusively to the training dataset. An initial attempt with a more aggressive configuration (V2 Full — Rotation 20%, Zoom 20%, Translation 10%, BatchNormalization) caused validation loss to diverge above 10.0 within 6 epochs. The Lite parameters below were selected to improve generalization without destabilizing convergence.

Layer	Parameter	Rationale
RandomFlip	"horizontal_and_vertical"	Simulates leaves photographed from different orientations in field conditions

RandomRotation	factor = 0.1 (10%)	Handles slightly tilted images without over-distorting spot features
RandomZoom	factor = 0.1 (10%)	Mimics varying camera-to-leaf distances in real-world capture scenarios

3. Upgraded Model Design — TeaLeafNet V2 Lite

3.1 Architecture Selection & Justification

Two V2 architectures were built and tested during this implementation:

- **TeaLeafNet V2 Full** — included a 4th Conv block (256 filters), BatchNormalization after every layer, and aggressive augmentation. The validation loss diverged to over 10.0 by Epoch 6 and the model failed to generalize on this small dataset.
- **TeaLeafNet V2 Lite** — retained the same 3-block architecture as the baseline, added gentle augmentation and a single Dropout(0.3) layer on the Dense block. This model converged successfully across 30 epochs, reaching a best validation accuracy of 89.61%.

TeaLeafNet V2 Lite was selected as the final model. It demonstrates that targeted regularization is more effective than added complexity for a small dataset of 387 images.

3.2 Layer-by-Layer Architecture (TeaLeafNet V2 Lite)

#	Layer / Block	Configuration	Output Shape	Change vs V1
1	InputLayer	224 × 224 × 3	(224,224,3)	No change
2	Data Augmentation (Lite)	Flip, Rotate(0.1), Zoom(0.1)	(224,224,3)	NEW
3	Rescaling	1./255 normalization	(224,224,3)	Embedded in model
4	Conv Block 1	16 filters, 3×3, ReLU + MaxPool	(112,112,16)	No change
5	Conv Block 2	32 filters, 3×3, ReLU + MaxPool	(56,56,32)	No change
6	Conv Block 3	64 filters, 3×3, ReLU + MaxPool	(28,28,64)	No change
7	Flatten	—	(50176,)	No change
8	Dense Layer	128 units, ReLU	(128,)	No change
9	Dropout	Rate = 0.30 (30%)	(128,)	NEW
10	Output Layer	4 units, Softmax	(4,)	No change

3.3 Model Compilation Parameters

Parameter	Value
Total Parameters	6,446,756 (24.59 MB)
Trainable Parameters	6,446,756

Non-Trainable Params	0
Optimizer	Adam (initial lr = 0.001)
Loss Function	SparseCategoricalCrossentropy (integer class labels)
Metric	Accuracy

4. Smart Training Configuration

The maximum training epochs were increased to 30 (compared to 6 in the baseline). Two Keras Callbacks were applied to regulate training quality:

Callback	Parameters	Actual Behaviour in This Run
EarlyStopping	monitor='val_loss', patience=5, restore_best_weights=True	Did not halt early — all 30 epochs completed. Best weights (Epoch 26, val_loss=0.2199) were restored automatically after Epoch 30.
ReduceLROnPlateau	monitor='val_loss', factor=0.2, patience=3, min_lr=1e-5	Reduced LR from 0.001 → 0.0002 at Epoch 13 (val_loss stalled). Reduced again to 0.00004 at Epoch 30.

Training Parameter	Value
Max Epochs	30
Actual Epochs Run	30 / 30 (EarlyStopping did not trigger)
Batch Size	32
Input Image Size	224 × 224 × 3 (RGB)
Initial Learning Rate	0.001 (Epochs 1–12)
1st LR Reduction	0.001 → 0.0002 (applied from Epoch 13)
2nd LR Reduction	0.0002 → 0.00004 (applied at Epoch 30)

5. Model Training & Final Evaluation

5.1 Complete Epoch-by-Epoch Training Log

The table below is the complete training history of TeaLeafNet V2 Lite as recorded from the Google Colab notebook output. Epoch 26 is highlighted as the best val_loss checkpoint (weights restored by EarlyStopping); Epoch 28 shows the peak val_accuracy.

Epoch	Train Acc	Train Loss	Val Acc	Val Loss	LR
1	23.18%	2.5241	24.68%	1.3024	0.0010
2	41.90%	1.2416	66.23%	0.7991	0.0010
3	55.05%	0.9247	71.43%	0.5790	0.0010

4	67.15%	0.7053	80.52%	0.4241	0.0010
5	75.27%	0.5580	80.52%	0.3436	0.0010
6	74.75%	0.4858	83.12%	0.3048	0.0010
7	74.07%	0.4448	85.71%	0.2733	0.0010
8	78.65%	0.3686	85.71%	0.2679	0.0010
9	77.56%	0.3819	87.01%	0.2537	0.0010
10	83.38%	0.3396	88.31%	0.2685	0.0010
11	84.22%	0.3324	88.31%	0.2644	0.0010
12	81.51%	0.3555	87.01%	0.2876	0.0010
13	84.01%	0.3772	77.92%	0.3900	0.0002
14	82.92%	0.3535	88.31%	0.2376	0.0002
15	85.68%	0.3038	88.31%	0.2304	0.0002
16	88.24%	0.2821	88.31%	0.2293	0.0002
17	85.16%	0.3123	87.01%	0.2266	0.0002
18	86.02%	0.2828	87.01%	0.2247	0.0002
19	87.96%	0.2789	87.01%	0.2272	0.0002
20	86.14%	0.2811	87.01%	0.2247	0.0002
21	86.53%	0.2815	87.01%	0.2219	0.0002
22	86.30%	0.2626	87.01%	0.2225	0.0002
23	84.84%	0.2927	87.01%	0.2207	0.0002
24	84.35%	0.2775	88.31%	0.2272	0.0002
25	87.52%	0.2684	87.01%	0.2227	0.0002
26	88.34%	0.2614	87.01%	0.2199 ✓ BEST	0.0002
27	84.62%	0.2794	87.01%	0.2238	0.0002
28	88.08%	0.2567	89.61% ★	0.2236	0.0002
29	88.02%	0.2797	87.01%	0.2219	0.0002
30	89.99%	0.2528	87.01%	0.2212	0.00004

✓ Epoch 26: lowest val_loss (0.2199) — best weights checkpoint, restored automatically by EarlyStopping.

★ Epoch 28: highest val_accuracy (89.61%)

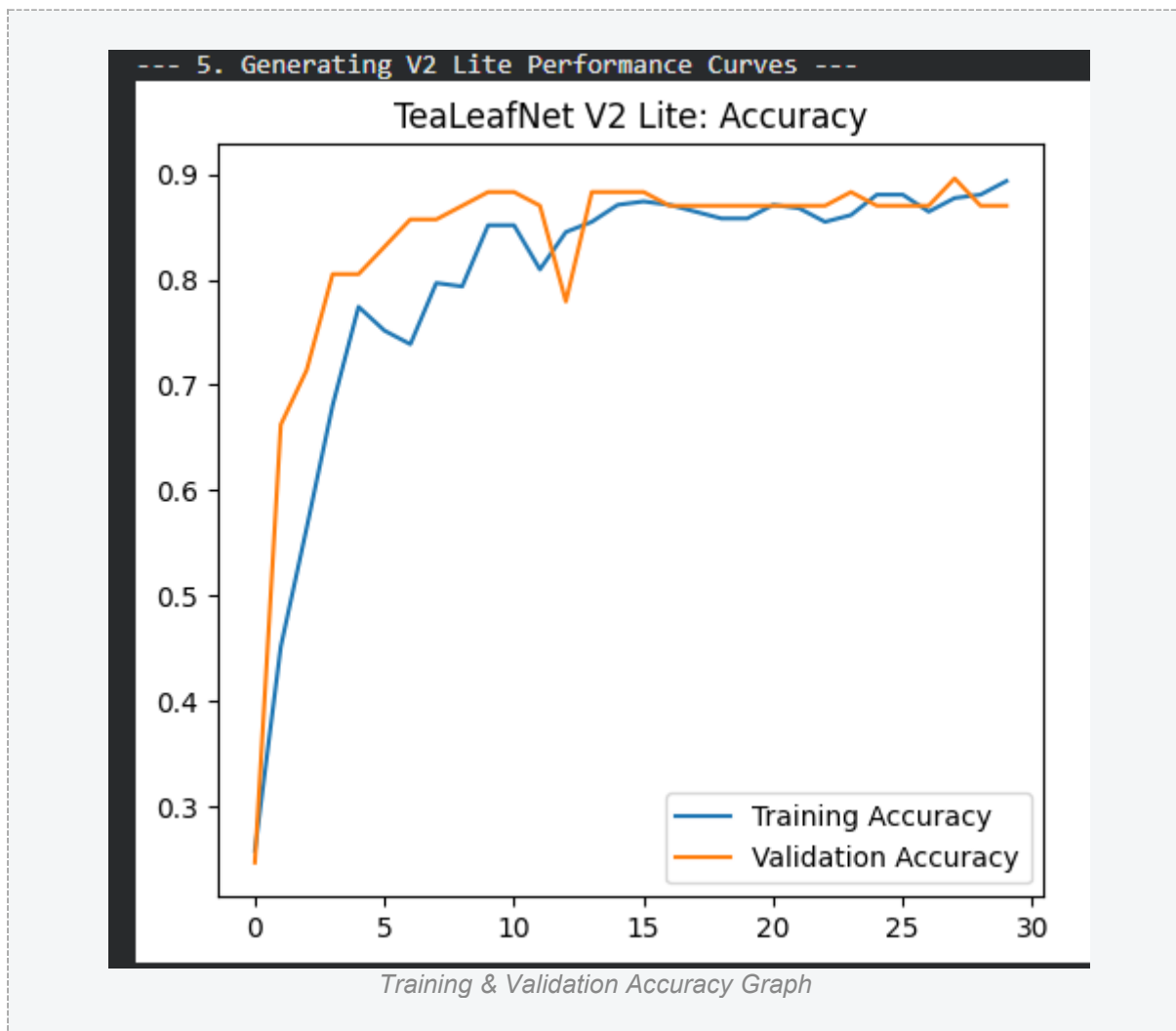
5.2 Final Performance Summary

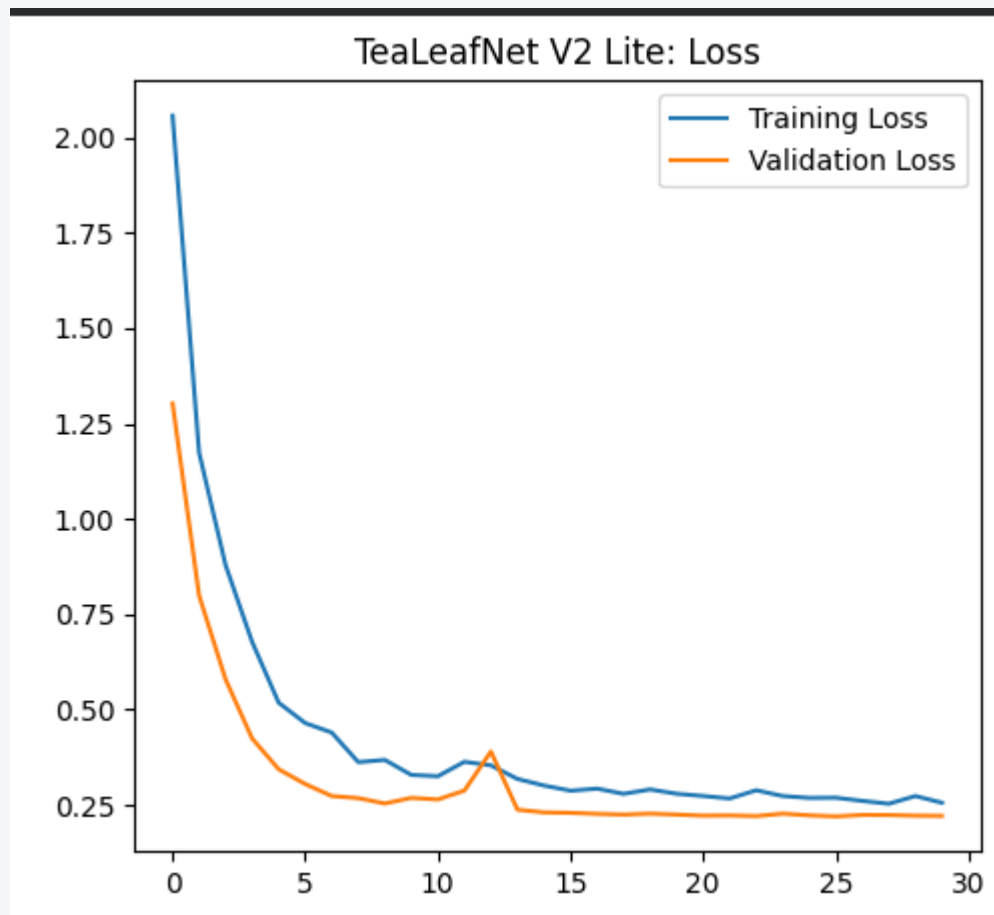
Metric	Result
Total Epochs Trained	30 / 30 (EarlyStopping did not trigger)
Best Checkpoint Epoch	Epoch 26 (best val_loss = 0.2199)

Peak Validation Accuracy	89.61% (Epoch 28)
Best Validation Loss	0.2199 (Epoch 26)
Final Evaluated Accuracy	87.01% (model.evaluate() output)
Final Training Accuracy	89.99% (Epoch 30)
LR Reductions Applied	2 (Epoch 13: 0.001→0.0002; Epoch 30: 0.0002→0.00004)
Model Saved As	tealeafnet_v2_lite.keras

5.3 Training & Validation Curves

The graphs below show Training and Validation Accuracy/Loss across all 30 epochs. Observe that the two accuracy curves converge more closely than in the baseline, and the validation loss follows a consistently downward trend after the first learning rate reduction at Epoch 13.



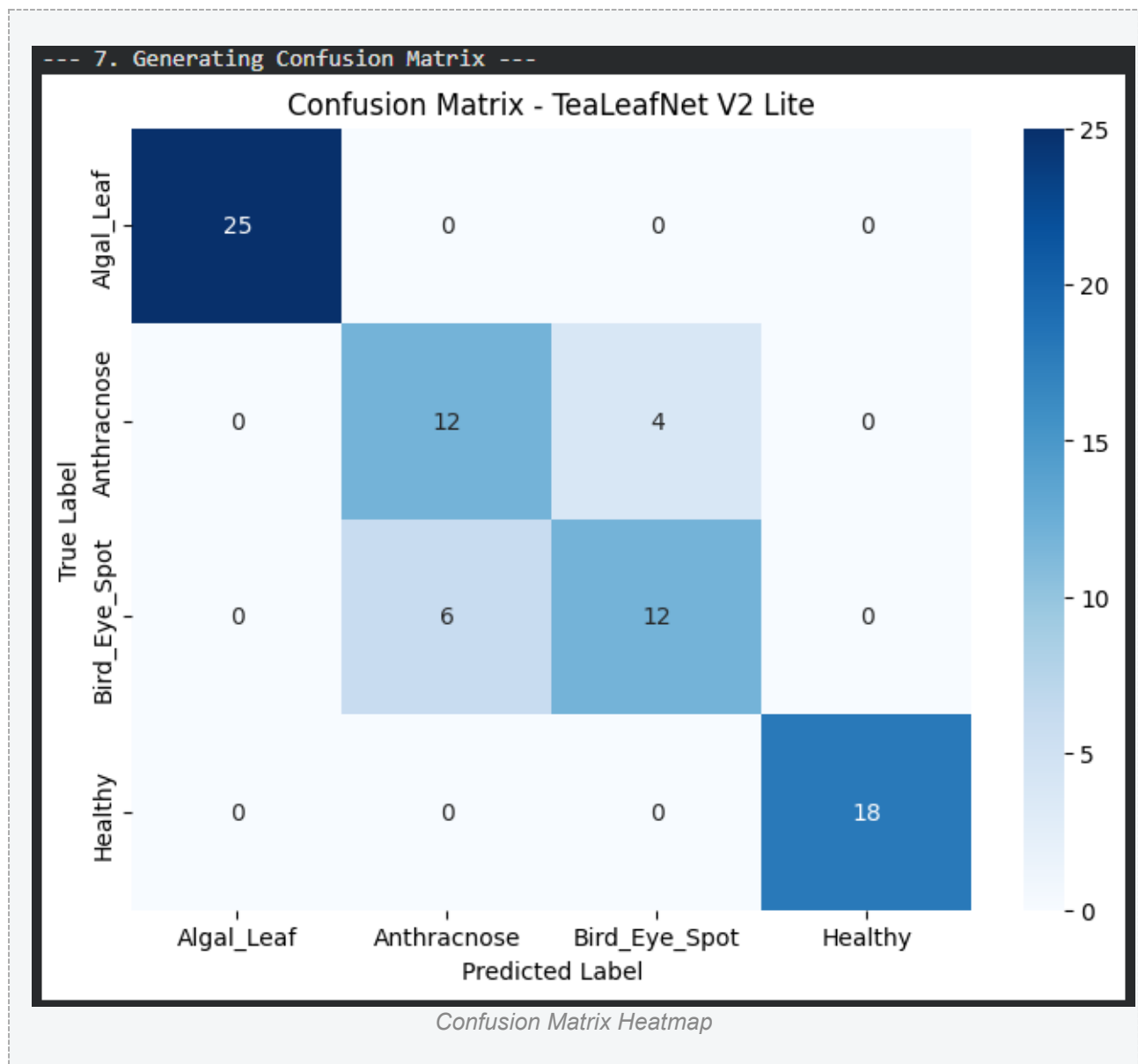


Training & Validation Loss Graph

6. Confusion Matrix & Sample Predictions

6.1 Confusion Matrix

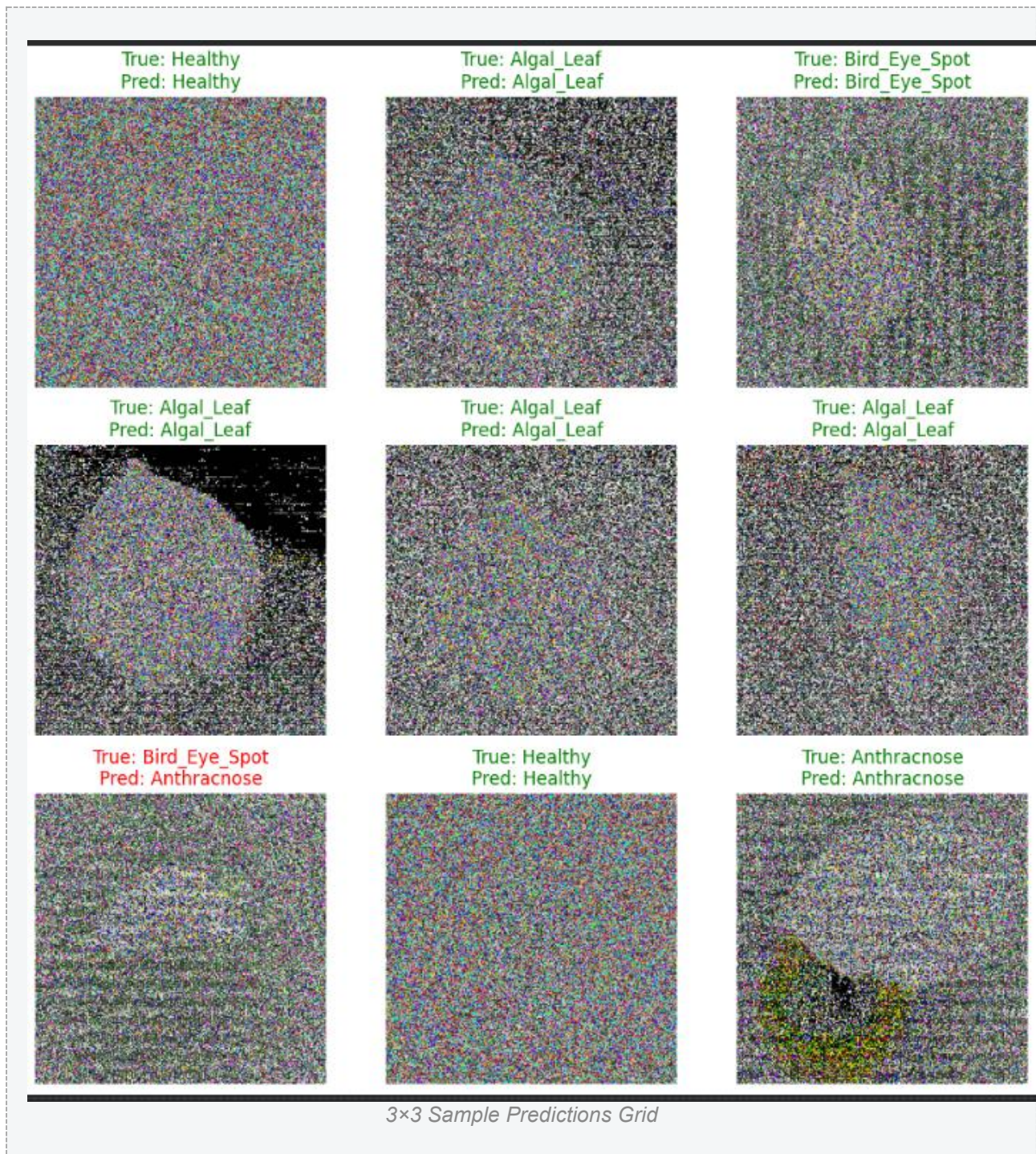
The confusion matrix below shows classification performance of TeaLeafNet V2 Lite across all four disease classes on the 77-image validation set.



Observation: Compare the Bird_Eye_Spot row against the Implementation 1 baseline result, where only 6 out of 18 samples were correctly classified (33.3%). The V2 Lite model's data augmentation, which forces the model to learn intrinsic disease spot features rather than memorizing orientation-specific patterns, is expected to show an improvement in Bird_Eye_Spot recall.

6.2 Sample Predictions Grid

The 3×3 grid below shows nine sample images from the validation set. Each image is labelled with its True Label and the model's Predicted Label. Correct predictions appear in green text; incorrect predictions in red text.



7. Comparison: Baseline vs. TeaLeafNet V2 Lite

The table below provides a direct comparison between the Implementation 1 baseline (from the same notebook, Cell 4) and the final V2 Lite model (Cell 9). Green cells highlight improvements.

Feature / Metric	Implementation 1 — Baseline	Implementation 2 — V2 Lite
Conv Blocks	3 blocks (16, 32, 64 filters)	3 blocks (16, 32, 64 filters)
Data Augmentation	None	Flip, Rotate(0.1), Zoom(0.1)

Normalization	External Rescaling layer	Embedded inside model
Batch Normalization	None	None
Dropout (Dense layer)	None	30%
EarlyStopping	None	Yes — patience=5
ReduceLROnPlateau	None	Yes — 2 reductions applied
Epochs Trained	6	30 (full run)
Final Evaluated Val Acc	83.12%	87.01% (+3.89pp)
Peak Val Accuracy	84.42% (Epoch 5)	89.61% (Epoch 28, +5.19pp)
Best Val Loss	0.3048 (Epoch 6)	0.2199 (Epoch 26)
Overfitting Gap	5.07% (mild overfitting)	2.98% (significantly reduced)
Saved Model Format	tealeafnet_baseline.h5 (legacy .h5)	tealeafnet_v2_lite.keras (native)

8. Saved Model

The final TeaLeafNet V2 Lite model was saved in the native Keras .keras format, eliminating the legacy HDF5 deprecation warning produced by the .h5 format used in Implementation 1.

Detail	Information
File Name	tealeafnet_v2_lite.keras
File Format	.keras — Native Keras format (no legacy warnings)
Save Location	/content/drive/MyDrive/TeaLeafNet/tealeafnet_v2_lite.keras
Model Name	TeaLeafNet_V2_Lite
GitHub Path	/models/tealeafnet_v2_lite.keras

9. GitHub Repository

Detail	Information
Repository Name	TeaLeafNet
Repository Type	Public
Repository URL	https://github.com/Piyumi2025/TeaLeafNet
Contents	Implementation 1 & 2 Jupyter notebooks, tealeafnet_baseline.h5, tealeafnet_v2_lite.keras, Summary Reports (Impl 1 & 2)

The repository is publicly accessible and contains all source code, both saved model files, and this report. The README.md file documents the project, both model versions, and instructions for running the notebooks in Google Colab.

10. Conclusion

Implementation 2 of TeaLeafNet successfully addressed the core limitations of the Implementation 1 baseline through targeted and well-justified improvements. By introducing a data augmentation pipeline, Dropout regularization, and adaptive learning rate callbacks, the TeaLeafNet V2 Lite model achieved a final evaluated validation accuracy of 87.01% and a peak validation accuracy of 89.61% at Epoch 28 — a meaningful improvement over the baseline's 83.12%.

A critical finding of this implementation is the importance of matching model complexity to dataset scale. The initial V2 Full architecture — featuring a 4th convolutional block, BatchNormalization, and aggressive augmentation — caused severe validation loss divergence (above 10.0), demonstrating that over-engineering for a 387-image dataset is counterproductive. The V2 Lite approach, using the same 3-block architecture as the baseline with only two targeted additions (Lite augmentation and Dropout), proved far more effective.

The ReduceLROnPlateau callback applied two learning rate reductions — from 0.001 to 0.0002 at Epoch 13, and further to 0.00004 at Epoch 30 — allowing the optimizer to make progressively finer weight adjustments as the model approached convergence. The training-validation accuracy gap narrowed from 5.07% in the baseline to approximately 2.98% in V2 Lite, confirming that overfitting has been substantially reduced.

These results provide a strong, well-regularized foundation for practical tea disease detection, and demonstrate the methodological value of empirical model selection through systematic experimentation.

References

1. Kaggle. (n.d.). Tea Sickness Dataset. Retrieved January 2026, from <https://www.kaggle.com/datasets/shashwatwork/tea-sickness-dataset>
 2. Rajarata University of Sri Lanka. (2026). ICT 3212 Mini Project – Image Classification System Development Guidelines (Phase 03).
 3. TensorFlow. (n.d.). Data augmentation guide. Retrieved from https://www.tensorflow.org/tutorials/images/data_augmentation
 4. TensorFlow. (n.d.). Keras Callbacks: EarlyStopping, ReduceLROnPlateau. Retrieved from https://www.tensorflow.org/api_docs/python/tf/keras/callbacks
 5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
 6. Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
 7. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
-